

Sistema de Gestión de Inventarios para Campesinos y Tiendas Locales de Valledupar

Proyecto de aula — Programación de Computadores III Universidad Popular del Cesar (UPC) — Valledupar, 2026

Autor: Bayron Enrique Lobo López **Docente:** Ing. Esp. Alfredo Bautista

Descripción del proyecto

Aplicación de escritorio desarrollada en Java con JavaFX, diseñada para apoyar a campesinos y tenderos de Valledupar en la digitalización del control de inventario, sustituyendo los registros manuales en cuadernos por un sistema confiable, sencillo y que funciona sin necesidad de conexión a Internet.

El sistema permite gestionar productos, categorías, proveedores, usuarios y movimientos de inventario, generando alertas automáticas de stock bajo y reportes sobre el estado del negocio.

Objetivo general

Desarrollar un sistema de información de escritorio en Java para la gestión de inventarios de campesinos y tiendas locales de Valledupar, aplicando principios de programación orientada a objetos, arquitectura en tres capas, patrones GRASP y SOLID, y persistencia de datos mediante el patrón DAO sobre una base de datos relacional.

Funcionalidades principales

- Gestión de usuarios con control de acceso por rol (administrador / operador)
 - Autenticación con contraseñas cifradas (SHA-256)
 - CRUD de categorías y proveedores
 - Registro y consulta de productos con filtros por categoría, proveedor y estado
 - Registro de movimientos de inventario (entrada, salida, ajuste) con trazabilidad completa
 - Alertas automáticas cuando el stock llega al mínimo configurado
 - Generación de reportes de inventario, movimientos y alertas
-

Arquitectura del sistema

El sistema está construido bajo una arquitectura en tres capas, con comunicación unidireccional:

Presentación (view) -> Negocio (service) -> Acceso a Datos (dao) -> Base de Datos

Capa	Paquete	Responsabilidades
Presentación	inventarios.valledupar.view	Ventanas JavaFX, formularios, manejo de eventos. Sin lógica de negocio
Negocio	inventarios.valledupar.service	Reglas del dominio, validaciones, coordinación entre capas.
Acceso a Datos	inventarios.valledupar.dao	PreparedStatement, ResultSet, mapeo objeto-relacional.
Modelo	inventarios.valledupar.model	Clases POJO que representan las entidades del dominio.
Conexión	inventarios.valledupar.dao.conexion	Singleton de conexión JDBC a Oracle XE.

Estructura de paquetes

```

inventarios.valledupar/
├── model/
│   ├── Persona (abstracta)
│   ├── Usuario
│   ├── Producto
│   ├── Categoria
│   ├── Proveedor
│   ├── Movimiento
│   ├── Alerta
│   └── Reporte
├── dao/
│   ├── IUserDAO, IProductoDAO, IMovimientoDAO,
│   │   IAlertaDAO, ICategoriaDAO, IProveedorDAO
│   ├── UsuarioDAO, ProductoDAO, MovimientoDAO,
│   │   AlertaDAO, CategoriaDAO, ProveedorDAO
│   └── conexion/ConexionBD (Singleton)
├── service/
│   ├── UsuarioService
│   ├── InventarioService
│   ├── AlertaService
│   ├── ReporteService
│   ├── CategoriaService
│   └── ProveedorService
└── view/
    ├── VentanaLogin
    ├── VentanaPrincipal
    ├── VentanaProductos / FormProducto
    ├── VentanaMovimientos / FormMovimiento
    └── VentanaAlertas
  
```

└─ VentanaReportes
└─ VentanaUsuarios
└─ VentanaProveedores

Modelo de clases

Clase	Tipo	Descripción
Persona	Abstracta	Atributos comunes: nombre, correo, teléfono. Método abstracto mostrarInfo().
Usuario	Concreta (extiende Persona)	Credenciales, rol y estado de la cuenta.
Producto	Concreta	Entidad central del inventario: precios, stock, unidad de medida.
Categoria	Concreta	Clasificación de productos.
Proveedor	Concreta	Información de contacto y estado de proveedores.
Movimiento	Concreta	Entradas, salidas y ajustes con trazabilidad de stock.
Alerta	Concreta	Notificación automática de stock bajo.
Reporte	Concreta	Resúmenes generados sobre el estado del inventario.

Base de datos

- Motor: Oracle Database 21c Express Edition (XE)
- Driver JDBC: ojdbc11.jar (oracle.jdbc.OracleDriver)
- URL de conexión: jdbc:oracle:thin:@localhost:1521:xe
- Patrón de conexión: Singleton (ConexionBD)

Tablas principales

USUARIOS, CATEGORIAS, PROVEEDORES, PRODUCTOS, MOVIMIENTOS, ALERTAS, REPORTES

Mecanismos de base de datos

Objeto	Nombre	Función
Trigger	TRG_GENERAR_ALERTA	Genera alerta automática cuando stock_actual <= stock_minimo.
Trigger	TRG_VALIDAR_SALIDA	Evita que una salida deje stock negativo.
Trigger	TRG_PROTEGER_PRODUCTO	Impide eliminar un producto activo. Asigna SYSDATE

Trigger	TRG_FECHA_MOVIMIENTO	automáticamente si la fecha es nula.
Paquete	PKG_INVENTARIO	Agrupar procedimientos: productos críticos, historial, atender alerta, desactivar producto.
Secuencias	SEQ_USUARIOS, etc.	Generación de claves primarias.
Índices	IDX_PRODUCTO_*, etc.	Optimización de consultas frecuentes.

Principios y patrones aplicados

Patrones GRASP

- Creator: InventarioService crea instancias de Movimiento y Alerta.
- Controller: UsuarioService e InventarioService coordinan operaciones entre capas.
- Expert: Producto conoce su propio estado (tieneStockBajo(), getMargenGanancia()).
- Low Coupling: arquitectura en capas e interfaces DAO.
- High Cohesion: AlertaService gestiona exclusivamente la lógica de alertas.
- Polymorphism: mostrarInfo() implementado de forma distinta en Usuario.

Principios SOLID

Principio	Aplicación
SRP	Cada DAO gestiona el acceso a datos de una sola entidad.
OCP	Las interfaces DAO permiten extender sin modificar controladores.
LSP	Usuario puede sustituir a Persona sin alterar el comportamiento esperado.
ISP	Interfaces DAO segregadas por entidad.
DIP	Los servicios dependen de interfaces DAO, no de implementaciones concretas.

Tecnologías utilizadas

- Lenguaje: Java
- IDE: NetBeans
- Interfaz gráfica: JavaFX (con CSS personalizado)

- Base de datos: Oracle Database 21c XE
 - Persistencia: JDBC con patrón DAO
 - Control de versiones: Git / GitHub (Conventional Commits)
-

Cómo ejecutar el proyecto

1. Tener instalado Oracle Database 21c XE y crear el esquema con los scripts SQL incluidos.
 2. Abrir el proyecto en NetBeans con el JDK y el SDK de JavaFX configurados.
 3. Verificar que el driver ojdbc11.jar esté agregado al classpath del proyecto.
 4. Ajustar las credenciales de conexión en ConexionBD si es necesario.
 5. Ejecutar la clase principal e iniciar sesión desde VentanaLogin.
-

Fases del proyecto

- Fase 1: Identificación del problema y especificación de requisitos
 - Fase 2: Definición de arquitectura y diseño de clases
 - Fase 3: Implementación y desarrollo
 - Fase 4: Documentación técnica final
-

Bibliografía

Blasco, F. (2022). Programación orientada a objetos. Ra-Ma.

Pino Herrera, J., Martínez Moreno, P. y Vergara, J. A. (2021). Fundamentos de la programación en Java: Estructuras de control e introducción a la POO. Universidad Nacional de Colombia.

Gervais, L. (2022). Aprender la programación orientada a objetos con el lenguaje Java. Ediciones ENI.

Martin, R. C. (2017). Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall.

Larman, C. (2004). Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development (3rd ed.). Prentice Hall.

Autor

Bayron Enrique Lobo López Desarrollador Principal / Líder del proyecto